

高级语言程序设计

实验报告

南开大学 计算机伯苓班

姓名 林永盛

学号 2511107

2026 年 5 月 13 日

目录

高级语言程序设计大作业实验报告	2
一、作业题目	3
二、开发软件	3
三、课题要求	3
四、游戏玩法与亮点介绍	3
五、主要流程	4
六、单元测试	4
七、AI 工具使用披露	6
八、总结与收获	
1. 内存管理的重要性	6

高级语言程序设计大作业实验报告

一、作业题目

熊出没类幸存者游戏：基于 C++ 与 EasyX 框架开发的面向对象双人同屏或单人类幸存者肉鸽生存游戏。

二、开发软件

- 1.IDE: Visual Studio 2022
- 2.图形库: EasyX Graphics Library

三、课题要求

1. 使用 C++ 语言完成一个图形化的小程序，图形化平台不限（本项目采用 EasyX）。
2. 程序内容主题不限，看重创新、创意。
3. 运用面向对象（OOP）编程思想。
4. 代码通过 Gitee/GitHub 托管，体现版本迭代。
5. 录制 B 站项目讲解与演示视频。

四、游戏玩法与亮点介绍

1. 游戏玩法

(1) 核心机制： 玩家操控角色在不断生成的敌群中生存，角色自动攻击，玩家需控制走位并寻找时机释放**专属技能**。

(2) 双人同屏： 支持本地双人合作， 分别通过 WASD 和方向键独立控制。

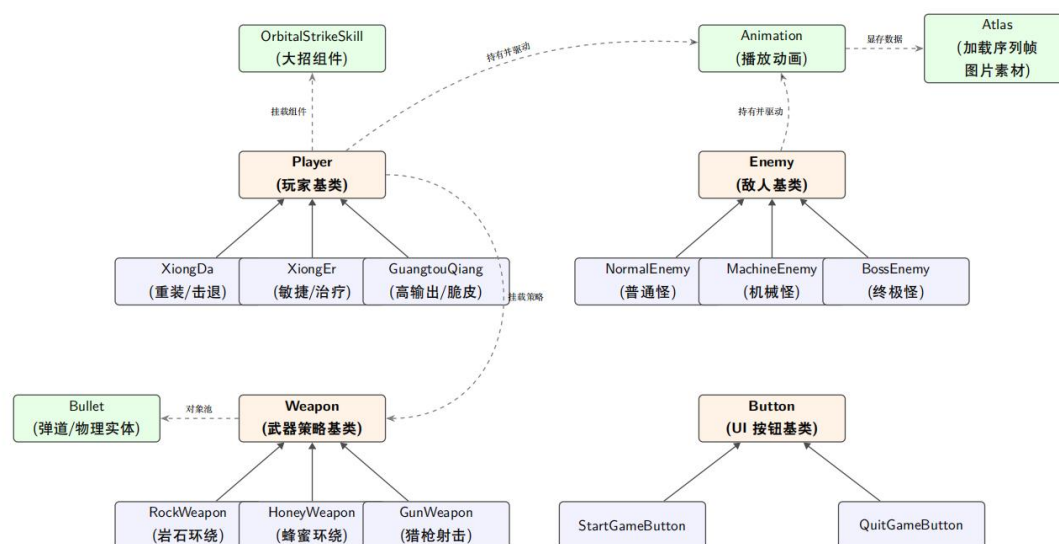
(3) 成长系统： 击杀敌人掉落经验球， 靠近时触发磁吸拾取。升级后触发“三选一”强化面板升级。

2. 核心系统实现

(1) 角色设计： 实现了三名机制互异的角色。熊大为重装定位（岩石环绕判定）；熊二为敏捷定位（蜂蜜罐武器）；光头强为射手定位（基于对象池管理的弹幕射击）。

(2) 游戏阶段： 游戏随存活时间推进提升难度， 游戏界面顶部有进度条， 进入不同阶段会有弹幕提醒。敌人从普通怪升级为高血量/移速的“机械怪”， 最终 Boss 包含定时召唤小怪机制。

五、主要流程



1. 整体架构 项目采用标准游戏主循环。系统状态分为主菜单（选人、难度切换）与战斗状态（实体更新、AI 追踪、碰撞结算、动画渲染）。

2. 核心算法与面向对象应用

(1)多态与虚基类： UI 系统通过抽象基类 Button 派生具体按钮，由 vector<Button*> 统一管理。敌人系统由 Enemy 基类派生出不同怪物，Boss 重写 Move() 函数加入召唤逻辑。利用虚析构函数防止内存泄漏。玩家由 player 基类派生出熊大熊二光头强三个子类，各自都重写了技能函数等。

(2)武器策略模式： 首先定义了一个 Weapon 抽象基类，再定义三个角色各自的武器类 RockWeapon、HoneyWeapon 和 GunWeapon。然后在各个角色的构造函数中使用基类指针指向子类对象为不同角色初始化各自的武器类，通过多态，主循环只需调用基类指针，就能执行不同的攻击逻辑，大大提高了代码的可维护性。

(3)寻路： 遍历存活玩家，锁定最近目标，计算方向向量并归一化，乘以速度系数实现匀速追踪。

六. 单元测试

输入	输出	测试目的
【熊二回血边界测试】 当前血量 140，最大血量 150。触发恢复 30%（45 点）生命技能。	hp 受阈值限制被截断为 150，无异常溢出。	验证角色技能数值及状态上限逻辑的安全边界。
【无序数组 O(1) 擦除性能	erase 遍历耗时引发卡顿；	验证对象池回收机制在极端

测试】 构造包含 100,000 个实体的无序 vector 数组，执行 10,000 次删除操作。	改用 swap + pop_back 优化后，耗时趋近于 0 毫秒。	压力下的性能表现。
---	------------------------------------	-----------

七、AI 工具使用披露

1. 使用的 AI 工具

- (1) Gemini 3.1 Pro (架构设计与算法优化)
- (2) 豆包 (素材生成)

2. 使用位置与具体用途

- (1) 算法优化：** 在子弹管理模块，参考 Gemini 建议应用了无序数组 Swap-Pop 删除策略，将时间复杂度降至 $O(1)$ 。此外，向 AI 请教了经验球磁吸缓动插值及武器轨迹的极坐标推导。
- (2) Bug 调试：** 配合 Gemini 定位了双人模式下的“幽灵特效”残留、基类缺乏虚析构函数引发的潜在内存泄露，以及伤害溢出表现等问题。
- (3) 资源生成：** 利用 AI 生成了部分素材图、背景原画。

八、总结与收获

1. 内存管理的重要性

本项目在堆区大量分配了实体对象。为了杜绝内存泄漏问题，在每一局游戏结束及程序退出时，均实现了统一的遍历删除逻辑 (delete 所有指针并 clear 容器)，加深了对 C++ 内存生命周期管理及虚析构函数必要性的理解。